

Functions - Answers

Note: Code solutions are provided in the `files/solution/` directory. This file contains explanations, PBL answers, and references to the solution files.

Answer: Functions 1 - Code Readability

Solution file: `files/solution/functions1_with_comments.c` (and `functions1_with_functions.c`)

Answers:

- What helps make code more readable without changing functionality:
 - **Meaningful variable names:** Replace cryptic names like `x`, `y`, `a` with descriptive names like `values_above_element`, `element_count`, etc.
 - **Comments:** Add comments to explain non-obvious code logic, complex algorithms, or business rules
 - **Consistent indentation:** Use consistent spacing and indentation throughout
 - **Proper spacing:** Add spaces around operators and after commas for better readability

PBL Answer: Meaningful variable names are important because:

1. **Self-documenting code:** Variables with good names explain what they represent without needing comments
2. **Maintainability:** When you or others revisit the code later, it's much easier to understand
3. **Debugging:** Clear names make it easier to identify what each variable does when tracking down bugs
4. **Collaboration:** Team members can understand your code more quickly
5. **Reduced errors:** Misleading names can cause misunderstanding and lead to bugs

Example: `N(211)=15` becomes much clearer when you know `N` means "count" and the parameters are meaningful.

Answer: Functions 2 - BMI Calculation Function

Solution file: `files/solution/functions2_sol.c`

Expected Output:

```
BMI: 1835.75
```

(For weight = 31.1, height = 1.31)

PBL Answer: Modified function to also print BMI category:

```
#include <stdio.h>

float calculateBMI(float weight, float height) {
    float bmi = weight / (height * height);

    // Print category
    if (bmi < 18.5) {
        printf("Category: Underweight\n");
    } else if (bmi >= 18.5 && bmi < 25.0) {
        printf("Category: Normal weight\n");
    } else if (bmi >= 25.0 && bmi < 30.0) {
        printf("Category: Overweight\n");
    } else {
        printf("Category: Obese\n");
    }

    return bmi;
}

int main() {
    float weight = 31.1;
    float height = 1.31;
    float bmi = calculateBMI(weight, height);
    printf("BMI: %.2f\n", bmi);
    return 0;
}
```

Answer: Functions 3 - Print Value with Pointer

Solution file: [files/solution/functions3_sol.c](#)

Expected Output:

```
address: 0x7ffd42a1b3ac, value: 101
```

(Address will vary each run)

Answers:

- What information is passed to `printValue`? The **memory address** of the variable `value` (a pointer to an int)

PBL Answer: Modified to print pointer size:

```
#include <stdio.h>

void printValue(int *pval) {
```

```
    printf("address: %p, value: %d\n", (void *)pval, *pval);
    printf("Size of pointer: %zu bytes\n", sizeof(pval));
}

int main() {
    int value = 101;
    printValue(&value);
    return 0;
}
```

Output will show the pointer size (typically 4 bytes on 32-bit systems, 8 bytes on 64-bit systems).

Answer: Patient 1 - Print Function (Call-by-Value)

Solution file: files/solution/patient1_sol.c

Expected Output:

```
Patient Information:
####
PatientID: 13
age:      44 years
weight:   80.0 kg
height:   1.80 m
```

PBL Answer: Complete function with all parameters:

```
#include <stdio.h>

void printPatientInfo(int id, int age, float weight, float height) {
    printf("Patient Information:\n");
    printf("###\n");
    printf(" PatientID:\t%d\n", id);
    printf(" age:\t\t%d years\n", age);
    printf(" weight:\t%.1f kg\n", weight);
    printf(" height:\t%.2f m\n", height);
}

int main() {
    int id = 13;
    int age = 44;
    float weight = 80;
    float height = 1.80;
    printPatientInfo(id, age, weight, height);
    return 0;
}
```

Answer: Patient 2 - Multiple Patients with BMI

Solution file: files/solution/patient2_sol.c

Expected Output:

```
Patient Information for 4 patients
###
PatientID: 1
age:      13 years
weight:   80.0 kg
height:   1.80 m
BMI:      24.69
####
PatientID: 3
age:      44 years
weight:   99.0 kg
height:   1.62 m
BMI:      37.87
... (for all 4 patients)
```

PBL Answer: Add BMI category printing:

```
#include <stdio.h>

float calculateBMI(float weight, float height) {
    return weight / (height * height);
}

void printPatientInfo(int id, int age, float weight, float height) {
    printf("###\n");
    printf(" PatientID:\t%d\n", id);
    printf(" age:\t\t%d years\n", age);
    printf(" weight:\t%.1f kg\n", weight);
    printf(" height:\t%.2f m\n", height);
}

int main() {
    int ids[] = {1, 3, 52, 23};
    int ages[] = {13, 44, 51, 100};
    float weights[] = {80, 99, 100, 50};
    float heights[] = {1.80, 1.62, 1.30, 1.74};
    int number_of_patients = sizeof(ids) / sizeof(ids[0]);

    printf("Patient Information for %d patients\n", number_of_patients);

    for (int i = 0; i < number_of_patients; i++) {
        printPatientInfo(ids[i], ages[i], weights[i], heights[i]);
        float bmi = calculateBMI(weights[i], heights[i]);
        printf(" BMI:\t\t%.1f\n", bmi);
    }
}
```

```
        // Print BMI category
        if (bmi < 18.5) {
            printf(" Category:\tUnderweight\n");
        } else if (bmi < 25.0) {
            printf(" Category:\tNormal weight\n");
        } else if (bmi < 30.0) {
            printf(" Category:\tOverweight\n");
        } else {
            printf(" Category:\tObese\n");
        }
    }
    return 0;
}
```

Answer: Records Display - Call-by-Reference

Solution file: `files/solution/records_display_sol.c`

Expected Output:

```
date: Fri Jul 17 10:23:00 2020
ID: 131
notes: Measurement recorded without issues
measurement: 93.30
```

PBL Answer: Extended to print multiple records:

```
#include <stdio.h>
#include <time.h>

struct Record {
    int id;
    time_t dato;
    char notes[100];
    float measurement;
};

void print_record(struct Record *r) {
    printf("date: %s", ctime(&r->dato));
    printf("ID: %d\n", r->id);
    printf("notes: %s\n", r->notes);
    printf("measurement: %.2f\n\n", r->measurement);
}

int main() {
    struct Record records[] = {
        {131, 1594974180, "Measurement recorded without issues", 93.3},
    }
```

```
        {132, 1595060580, "Second measurement", 95.5},  
        {133, 1595146980, "Third measurement", 91.2}  
    };  
  
    int num_records = sizeof(records) / sizeof(records[0]);  
  
    for (int i = 0; i < num_records; i++) {  
        print_record(&records[i]);  
    }  
  
    return 0;  
}
```

Answer: Patient 3 - Statistics Functions

Solution file: `files/solution/patient3_sol.c`

Expected Output:

```
Patient Descriptives for 4 Patients  
Age: mean = 52.00, std = 36.66  
Weight: mean = 82.25, std = 21.80  
Height: mean = 1.62, std = 0.21
```

(Exact values may vary based on implementation)

PBL Answer: Calculate all statistics in one loop:

```
#include <stdio.h>  
#include <math.h>  
  
void printStatistics(char* measurement, float mean, float std) {  
    printf("%s: mean = %.2f, std = %.2f\n", measurement, mean, std);  
}  
  
float mean(float x[], int n) {  
    float sum = 0;  
    for (int i = 0; i < n; i++) {  
        sum += x[i];  
    }  
    return sum / n;  
}  
  
float std(float x[], int n) {  
    float m = mean(x, n);  
    float sum_sq = 0;  
    for (int i = 0; i < n; i++) {  
        sum_sq += pow(x[i] - m, 2);  
    }  
    return sqrt(sum_sq / n);  
}
```

```
    }
    return sqrt(sum_sq / (n - 1));
}

int main() {
    int ages[] = {13, 44, 51, 100};
    float weights[] = {80, 99, 100, 50};
    float heights[] = {1.80, 1.62, 1.30, 1.74};
    int number_of_patients = sizeof(ages) / sizeof(ages[0]);

    printf("Patient Descriptives for %i Patients\n", number_of_patients);

    // Calculate and print statistics for all three measurements
    printStatistics("Age", mean_int(ages, number_of_patients),
std_int(ages, number_of_patients));
    printStatistics("Weight", mean(weights, number_of_patients),
std(weights, number_of_patients));
    printStatistics("Height", mean(heights, number_of_patients),
std(heights, number_of_patients));

    return 0;
}
```

File References

All answer files are located in: [files/solution/](#)

- Functions 1 - Code Readability: [functions1_with_comments.c](#) and [functions1_with_functions.c](#)
- Functions 2 - BMI Calculation: [functions2_sol.c](#)
- Functions 3 - Print Value with Pointer: [functions3_sol.c](#)
- Patient 1 - Print Function: [patient1_sol.c](#)
- Patient 2 - Multiple Patients with BMI: [patient2_sol.c](#)
- Records Display - Call-by-Reference: [records_display_sol.c](#)
- Patient 3 - Statistics Functions: [patient3_sol.c](#)